

Day 7 — API Fundamentals & Postman

API Exploration Notes: inspecting real requests and responses before wiring anything into n8n

Intern Name	Musadiqa Fiaz
University / Institute	Hazara University
Internship Domain	AI Automation
Mode	Online / Remote
Organization	Petalnex Pvt. Ltd.
Supervisor / Team Lead	Sohail Anwar

1. Core Concepts

What is an API?

An API (Application Programming Interface) is a defined way for one piece of software to ask another piece of software to do something or hand over data, without either side needing to know how the other is built internally. A web API exposes this over HTTP.

REST

REST (Representational State Transfer) is an architectural style for web APIs. A RESTful API models the system as a set of **resources** (e.g. a post, a user, a comment), each identified by a URL, and uses standard HTTP methods to act on those resources. Responses are usually JSON.

Endpoint / Request / Response

An **endpoint** is a specific URL that represents one resource or action (e.g. `/posts/1`). A **request** is what the client sends to that endpoint — it has a method, headers, optional query parameters, and an optional body. A **response** is what the server sends back — a status code, headers, and usually a body (JSON in the examples below).

2. HTTP Methods

Method	Purpose	Body?	Typical use
GET	Retrieve a resource or list	No	Fetch a post, list all posts
POST	Create a new resource	Yes	Create a new post/comment
PUT	Replace a resource entirely	Yes	Overwrite a post with a full new object

PATCH	Partially update a resource	Yes	Update just the title of a post
DELETE	Remove a resource	No	Delete a post

3. Status Codes

Code	Meaning	When it shows up
200 OK	Success	GET/PUT/PATCH succeeded, response body included
201 Created	Resource created	POST succeeded, new resource returned
400 Bad Request	Client sent malformed input	Missing/invalid field, broken JSON
401 Unauthorized	Missing or invalid credentials	No API key / token supplied
403 Forbidden	Authenticated but not allowed	Valid credentials, insufficient permission
404 Not Found	Resource doesn't exist	Wrong ID, wrong path, typo in URL
429 Too Many Requests	Rate limit exceeded	Too many calls in a short window
500 Internal Server Error	Server-side failure	Bug or crash on the API's server

4. Headers, Query Parameters, Request Bodies

Headers carry metadata about the request/response — e.g. Content-Type tells the server what format the body is in, Authorization carries credentials. **Query parameters** are key/value pairs appended to the URL after ? (e.g. ?postId=1) used mainly to filter or sort GET requests. **Request bodies** carry the actual data being sent, typically as JSON, and are used with POST, PUT and PATCH.

5. Practical Task — Live API Inspection

Tooling: requests were sent and inspected using **Postman** (collection: "API Exploration with Postman"). API used: **JSONPlaceholder** (<https://jsonplaceholder.typicode.com>) — a free fake REST API commonly used for practicing API calls; no key or signup required.

Request 1 — GET a single resource

URL: <https://jsonplaceholder.typicode.com/posts/1>

Method: GET

Headers sent: Accept: application/json (default browser/client headers; no auth required)

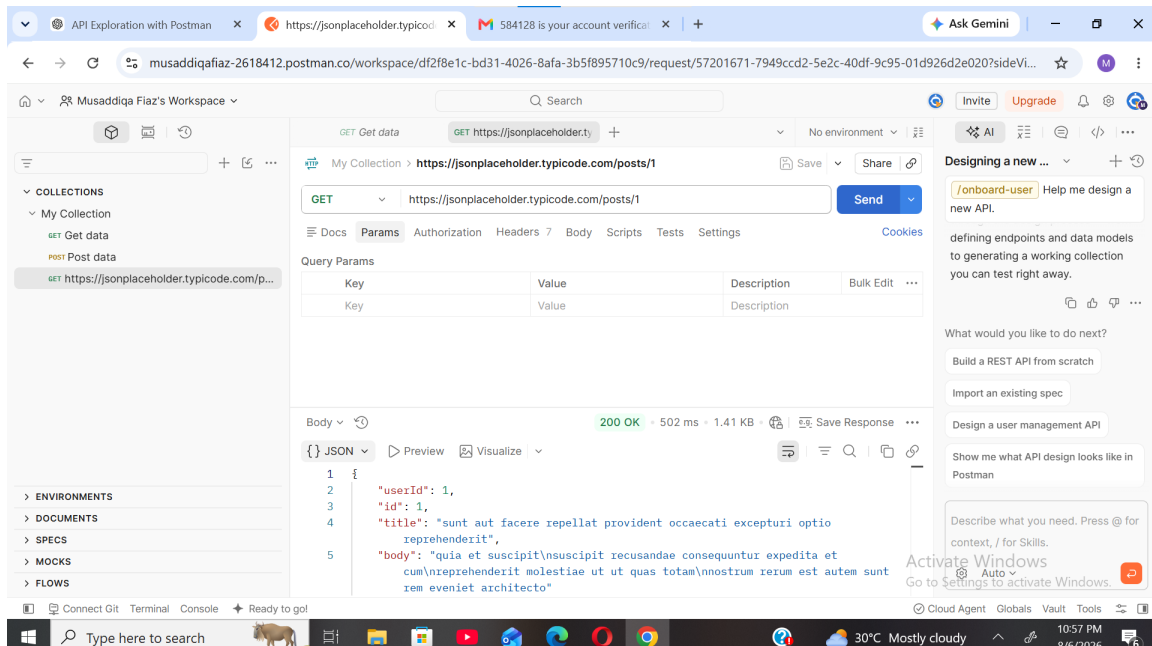
Response — Status: 200 OK

Response headers (key ones): Content-Type: application/json; charset=utf-8

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
}
```

Structure notes: The response is a single flat JSON object (not an array) representing one resource. Four fields: `userId` (int, foreign key pointing at a user), `id` (int, this resource's own ID — matches the 1 in the URL), `title` (string), `body` (string, contains literal `\n` line breaks). No nesting, no pagination — this is the simplest possible REST response shape.

Postman screenshot — request & response:



Postman panel shows: method GET, URL <https://jsonplaceholder.typicode.com/posts/1>, status **200 OK**, response time 502 ms, body size 1.41 KB, and the JSON body rendered in the Body/JSON tab.

Request 2 — GET with a query parameter (filtering)

URL: <https://jsonplaceholder.typicode.com/comments?postId=1>

Method: GET

Query parameter: postId=1 — filters the /comments collection down to only comments belonging to post 1

Response — Status: 200 OK

```
[
  {
    "postId": 1,
    "id": 1,
    "name": "id labore ex et quam laborum",
    "email": "Eliseo@gardner.biz",
    "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos..."
  },
  {
    "postId": 1,
    "id": 2,
    "name": "quo vero reiciendis velit similique earum",
    "email": "Jayne_Kuhic@sydney.com",
    "body": "est natus enim nihil est dolore omnis voluptatem numquam..."
  },
  ... 3 more comment objects (5 total for postId=1) ...
]
```

Structure notes: This time the top-level response is a JSON **array** of objects, because the endpoint returns a collection. Every object has the same shape: `postId` (which post it belongs to — a relation), `id` (the comment's own ID), `name`, `email`, `body`. This is the pattern to expect from any 'list' endpoint: array in, consistent object shape per item.

Request 3 — Triggering an error response

URL: <https://jsonplaceholder.typicode.com/invalid>

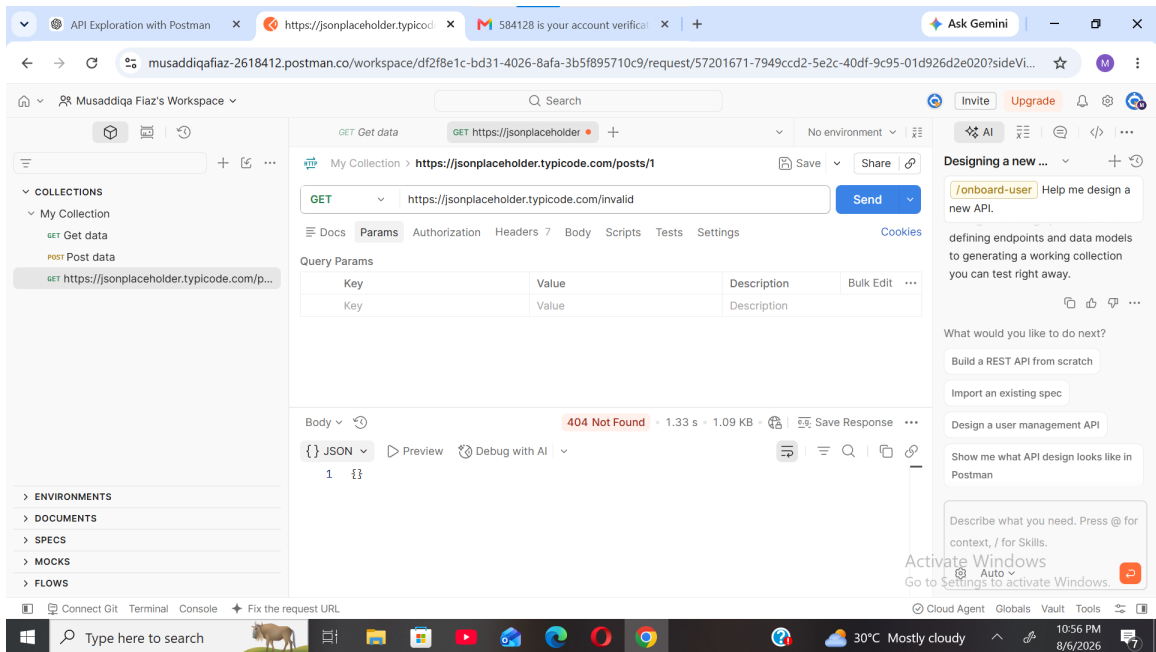
Method: GET

Why it errors: /invalid is not a real resource path on this API — only /posts, /comments, /albums, /photos, /todos and /users exist

Response — Status: 404 Not Found

```
{}
```

Postman screenshot — request & response:



Postman panel shows: method GET, URL `https://jsonplaceholder.typicode.com/invalid`, status **404 Not Found**, response time 1.33 s, body size 1.09 KB, and an empty JSON object `{}` in the body.

Notes: Instead of a 500-style crash, the server correctly reports 404 to say "this resource doesn't exist" and returns an empty JSON object rather than the usual post shape. This is the expected, well-behaved way for an API to fail — the status code alone is enough to tell a client (or an n8n workflow) that the request failed, without needing to parse the body. Worth noting the response time here (1.33s) is also noticeably slower than the successful request (502ms) — errors don't always come back fast.

6. Takeaways for n8n Integration

- Always check the response status code first in an n8n HTTP Request node — don't assume 200.
- A list endpoint returns an array; a single-resource endpoint returns an object. n8n's 'Split Out' step needs to know which shape it's getting.
- Query parameters are a cheap way to filter server-side instead of pulling everything and filtering in n8n afterward.
- A 404 with an empty body is still valid JSON — downstream nodes need to handle 'success but empty' as well as outright failures (401/403/429/500).
- Headers matter even without auth here — Content-Type on requests with a body, Accept on requests expecting JSON back.